

**STREDNÁ PRIEMYSELNÁ ŠKOLA
ELEKTROTECHNICKÁ
PREŠOV**

IV.SC – 25

2025 – 2026

**GRAFICKÁ APLIKÁCIA
SIEŤOVEJ HRY**

ROČNÍKOVÝ PROJEKT (RPJ + PČOZ)

ŠIMON ŠROBÁR

Zadanie vlastného projektu

pre ročníkový projekt (RPJ)

a praktickú časť odbornej zložky (PČOZ) maturitnej skúšky



Trieda: **IV.SC**

Školský rok: **2025 - 2026**

Názov: **Grafická aplikácia sieťovej hry**

Autor: **Šimon Šrobár**

Konzultant: **Ing. Mária Hedvigová**

Učiteľ RPJ: **RNDr. Martin Šechný, PhD.**

Úlohy:

1. Navrhnete funkčnosť grafickej aplikácie sieťovej hry Sedma berie tri.
2. Navrhnete a vytvorte atraktívne grafické prostredie hry a objekty, ktoré umiestnite do prostredia hry.
3. Vytvorte funkčnú grafickú aplikáciu hry s počítačom, ktorá bude fungovať podľa návrhu.
4. Realizujte možnosť hrať hru viacerým hráčom po počítačovej sieti.
5. Navrhnete a realizujete motiváciu hráčov hrať hru a zlepšovať sa.
6. Otestujete funkčnosť vlastnej aplikácie sieťovej hry.
7. Zhodnotíte výsledky svojej práce a porovnáate svoju aplikáciu s podobnými aplikáciami.
8. Vytvorte technickú dokumentáciu projektu podľa šablóny v digitálnej a papierovej forme.
9. Vytvorte prezentáciu projektu v digitálnej forme.
10. Zbalíte všetky súbory – dokumentáciu, prezentáciu a prílohy (napr. dáta, zdrojový kód programu, vzor plošného spoja, schému zapojenia, multimédiá) a odovzdajte ich spolu s papierovou dokumentáciou.

Dokumentácia projektu spolu so všetkými prílohami sa považuje za školské dielo podľa § 93 zákona 185/2015 Z. z. v aktuálnom znení (autorský zákon). Škola predpokladá uplatnenie verejnej licencie na školské dielo podľa § 76 tohto zákona. Môže byť uvedená konkrétna licencia (napr. CC-BY, EUPL, GNU LGPL, GNU GPL).

Cieľ projektu

Vytvoriť funkčnú grafickú sieťovú aplikáciu kartovej hry Sedma berie tri v programovacom jazyku Python pre 2 až 4 hráčov, ktorá umožní synchronizované hranie po lokálnej sieti s podporou viacerých herných miestností, turnajového režimu, dedikovaného servera a prispôsobiteľného grafického rozhrania, navrhnutú a otestovanú v priebehu ôsmich mesiacov v rámci študijného odboru informačné a sieťové technológie.

Čiastkové ciele – plán práce

Doplňte konkrétne plánované činnosti pri riešení zadania a konkrétne časti produktu:

- 09.2025 Zvoliť vhodný jazyk, knižnice a vývojové prostredie.
- 10.2025 Naštudovať problematiku sieťového programovania a pravidiel hry.
- 11.2025 Navrhnuť architektúru systému, implementovať hernú logiku a model karty.
- 12.2025 Implementovať sieťový server s podporou viacerých miestností a klientske GUI.
- 01.2026 Prepojiť klientov cez sieť, implementovať lobby a synchronizáciu herného stavu.
- 02.2026 Dokončiť dizajn, pridať prispôsobenie tém, animácie a turnajový režim.
- 03.2026 Pridať možnosť online hrania s logovaním a správou časových limitov. Otestovať a optimalizovať.
- 04.2026 Dokončiť dokumentáciu, prezentáciu a prílohy.

Úroveň pripravenosti technológie

Počiatočná úroveň: TRL 6

Cieľová úroveň: TRL 7

Čestné vyhlásenie

Čestne vyhlasujem, že celú dokumentáciu k projektu som vypracoval samostatne, s použitím uvedenej literatúry a s použitím uvedených nástrojov.

Prešov, 30.04.2026

.....

vlastnoručný podpis

Abstrakt

Grafická aplikácia sieťovej hry

Šrobár, Šimon

Projekt sa zameriava na vývoj grafickej sieťovej aplikácie kartovej hry Sedma berie tri v programovacom jazyku Python pre 2 až 4 hráčov. Aplikácia je implementovaná ako klient-server systém s podporou viacerých herných miestností, turnajového režimu a dedikatovaného servera určeného pre verejný hosting. Sieťová komunikácia využíva TCP sokety s JSON správami komprimovanými knižnicou zlib a odosielanými so 4-bajtovou hlavičkou. Grafické rozhranie je vytvorené pomocou knižnice Pygame a obsahuje animácie, prispôsobiteľné témy kariet a pozadí, ako aj rebríček výsledkov. Herná logika je oddelená do samostatného modulu a implementuje špeciálne pravidlá pre karty ako sedmičky, esá, horníci a zelený dolník. Server podporuje neblokujúce spracovanie viacerých klientov pomocou modulu selectors a zabezpečuje konzistentný herný stav pre všetkých hráčov. Testovanie prebiehalo v lokálnej sieti so štyrmi zariadeniami a potvrdilo dosiahnutie priemernej latencie pod 90 ms pri úspešnosti spojenia nad 97 %.

Abstract

Graphical Network Game Application

Šrobár, Šimon

The project focuses on developing a graphical network application for the card game Sedma berie tri in Python for 2 to 4 players. The application is implemented as a client-server system with support for multiple game rooms, tournament mode, and a dedicated server for public hosting. Network communication uses TCP sockets with JSON messages compressed using the zlib library and sent with a 4-byte header. The graphical interface is built with Pygame and includes card animations, customizable card and background themes, and a leaderboard system. Game logic is separated into a dedicated module and implements special rules for cards such as sevens, aces, jacks, and the green knave. The server handles multiple clients simultaneously using non-blocking I/O via the selectors module and maintains a consistent game state for all

players. Testing was conducted on a local network with four devices, confirming average latency below 90 ms with a connection success rate above 97%.

Obsah

Zadanie vlastného projektu.....	2
Cieľ projektu.....	3
Čiastkové ciele – plán práce	3
Úroveň pripravenosti technológie	3
Čestné vyhlásenie	4
Abstrakt	5
Úvod	10
1 Teoretická časť.....	12
1.1 Charakteristika hry Sedma berie tri	12
1.2 Počítačové siete a protokol TCP.....	12
1.3 Klient-server architektúra	13
1.4 Neblokujúce I/O a modul selectors	13
1.5 Serializácia dát a kompresia	13
1.6 Programovací jazyk Python.....	14
1.7 Grafická knižnica Pygame.....	14
1.8 Viacvláknové programovanie	15
2 Praktická časť	16
2.1 Návrh architektúry systému.....	16
2.2 Implementácia hernej logiky (game_logic.py).....	17
2.3 Sieťová komunikácia a server (server.py)	19
2.4 Dedicated server (dedicated_server.py).....	20
2.5 Klientská aplikácia a grafické rozhranie (client.py).....	20
2.6 Systém herných miestností a lobby	21
2.7 Turnajový režim.....	22
2.8 Prispôsobenie grafického prostredia.....	22
2.9 LAN server zabudovaný v klientovi.....	23
2.10 Testovanie aplikácie	23
2.11 Bezpečnosť a stabilita.....	24
2.12 Rozpočet projektu.....	24
Záver.....	25
Použitá literatúra.....	26
Použité nástroje	27

Prílohy	28
---------------	----

Úvod

V súčasnej dobe, keď digitálne technológie prestupujú každodenným životom, narastá záujem o prenesenie tradičných stolových hier do počítačového prostredia. Kartová hra Sedma berie tri patrí medzi obľúbené slovenské rodinné hry, ktoré doteraz absentujú v podobe modernej sieťovej aplikácie dostupnej pre lokálne siete. Práve táto medzera na trhu sa stala motiváciou pre vznik tohto projektu.

Existujúce riešenia online kartových hier sú väčšinou navrhnuté pre veľké internetové platformy s cloudovými servermi, čo prináša nevýhody v podobe vyššej latencie, nutnosti registrácie a závislosti od internetového pripojenia. Pre skupinu hráčov v jednej domácnosti alebo triede je takéto riešenie zbytočne komplikované. Cieľom projektu je preto vytvoriť jednoduchú, rýchlu a bezplatnú alternatívu určenú pre lokálnu sieť.

Hlavným cieľom projektu je vytvoriť funkčnú grafickú sieťovú aplikáciu kartovej hry Sedma berie tri v programovacom jazyku Python pre 2 až 4 hráčov s podporou viacerých herných miestností, turnajového režimu a dedikovaného servera. Aplikácia má dosahovať priemernú latenciu pod 100 ms a úspešnosť sieťového spojenia nad 95 %.

Projekt je rozdelený do viacerých čiastkových cieľov. V prvej fáze bola navrhnutá architektúra systému a implementovaná herná logika. V druhej fáze vzniklo grafické používateľské rozhranie, správa hráčov a systém herných miestností. Tretia fáza priniesla rozšírenia ako turnajový režim, dedikovaný server a prispôsobiteľné grafické témy. Záverečná fáza bola venovaná testovaniu, optimalizácii a tvorbe dokumentácie.

Cieľová skupina projektu sú hráči vo veku od 10 do 50 rokov, ktorí hľadajú rýchlu digitálnu zábavu v malej skupine bez potreby internetového pripojenia. Pridanou hodnotou oproti existujúcim riešeniam je lokálnosť, jednoduchosť pripojenia bez registrácie, open-source charakter a možnosť rozšírenia na ďalšie platformy.

Hypotéza projektu predpokladá, že sieťová aplikácia Sedma berie tri implementovaná pomocou TCP soкетов, zlib kompresie a neblokujúceho servera dosiahne v testovacom

prostredí lokálnej siete priemernú latenciu pod 90 ms a úspešnosť spojenia nad 97 % pri štyroch súčasne pripojených hráčoch.

1 Teoretická časť

Táto kapitola opisuje odborné východiská pre návrh a implementáciu sieťovej kartovej hry Sedma berie tri. Zameriava sa na charakteristiku hry, princípy počítačových sietí, použité technológie a teoretické základy sieťového a grafického programovania.

1.1 Charakteristika hry Sedma berie tri

Kartová hra Sedma berie tri patrí medzi tradičné stredoeurópske stolové hry určené pre 2 až 4 hráčov. Hrá sa s 32-kartovým balíčkom, ktorý obsahuje karty hodnôt 7 až Eso (Eso má hodnotu 14) v štyroch farbách: srdce, zeleň, žalud' a guľa. Cieľom hry je čo najrýchlejšie zahrať všetky karty z ruky.

Hráč môže zahrať kartu rovnakej farby alebo rovnakej hodnoty ako vrchná karta na odhadzovacom balíčku. Špeciálne karty menia priebeh hry: sedmička (7) donúti nasledujúceho hráča potiahnuť tri karty, pokiaľ ju neprebije ďalšou sedmičkou alebo zeleným dolníkom; eso (14) preskočí nasledujúceho hráča; horník (12) umožní hráčovi zmeniť farbu; zelený dolník (11 zeleň) môže prebiť aktívnu sedmičkovú penalizáciu. Pravidlá môžu byť upravené pri vytváraní hernej miestnosti, čo pridáva variabilitu.

Správne definované pravidlá validácie ťahov sú nevyhnutné, aby implementovaný algoritmus zabránil nelegálnym ťahom a udržal konzistentný stav hry pre všetkých hráčov.

1.2 Počítačové siete a protokol TCP

Lokálna sieť (LAN – Local Area Network) predstavuje komunikačnú infraštruktúru s obmedzeným geografickým rozsahom, typicky v jednej budove alebo domácnosti. Komunikácia prebieha prostredníctvom sieťových protokolov z rodiny TCP/IP. Protokol TCP (Transmission Control Protocol) zabezpečuje spoľahlivý prenos dát, garantované doručenie, zachovanie poradia paketov a detekciu chýb. Tieto vlastnosti ho robia vhodným pre aplikácie vyžadujúce presnosť a konzistenciu dát, ako sú sieťové hry. [3]

Latencia označuje čas potrebný na prenos správy medzi klientom a serverom a späť. Pre plynulé interaktívne hranie je žiadaná latencia nižšia ako 100 ms, aby nedochádzalo k viditeľným oneskoreniam. Šírka pásma (bandwidth) určuje maximálne množstvo dát

prenesených za sekundu. Pre aplikáciu s malým počtom hráčov a komprimovanými správami nie je šírka pásma kritickým faktorom, dôležitejšia je nízka latencia. [2]

1.3 Klient-server architektúra

Klient-server architektúra patrí medzi najpoužívanejšie modely pre sieťové aplikácie. Server predstavuje centrálny riadiaci uzol, ktorý spracúva požiadavky klientov, udržiava autoritatívny herný stav a distribuuje aktualizácie všetkým pripojeným hráčom. Klienti komunikujú so serverom prostredníctvom správ a zobrazujú aktuálny stav hry. Tento model zaručuje konzistenciu dát a predchádza konfliktom, ktoré by vznikali pri decentralizovanom riadení. [11]

Dôležitou vlastnosťou autoritatívneho servera je, že herné rozhodnutia sú spracované výhradne na serveri. Klient odosiela iba požiadavky na vykonanie akcií, server ich validuje a výsledok distribuuje všetkým hráčom. Týmto prístupom sa predchádza podvádzaniu a nekonzistentnému stavu hry.

1.4 Neblokujúce I/O a modul selectors

Pri obsluhu viacerých klientov súčasne existujú dve základné stratégie: vlákna (threading) a neblokujúce I/O (non-blocking I/O). Vlákňový prístup vytvára pre každého klienta samostatné vlákno, čo je jednoduché, ale pri väčšom počte klientov môže byť pamäťovo a výpočtovo náročné. Neblokujúce I/O pomocou mechanizmu select/poll/epoll umožňuje jednému vláknu monitorovať stovky soketov súčasne. [4]

Python ponúka štandardný modul selectors, ktorý poskytuje prenosnú abstrakciu nad systémovými volaniami select a epoll. Server registruje sokety do selektora a v hlavnej slučke čaká na udalosti čítania alebo zápisu. Tento prístup výrazne znižuje spotrebu pamäte a je vhodný pre sieťové servery s väčším počtom súbežných klientov. [4]

1.5 Serializácia dát a kompresia

Pre prenos herného stavu cez sieť je potrebné previesť dátové štruktúry do bajtového formátu (serializácia). JSON (JavaScript Object Notation) je ľahko čitateľný textový formát, ktorý Python natívne podporuje prostredníctvom modulu json. Je vhodný pre hierarchické dátové štruktúry a ľahko sa debuguje. [1]

Pre zníženie veľkosti prenášaných správ je možné použiť kompresiu. Knižnica `zlib` (modul `zlib` v Pythone) implementuje algoritmus DEFLATE, ktorý kombinuje LZ77 kompresiu a Huffmanovo kódovanie. JSON správy sa po kompresii môžu zmenšiť na 10–30 % pôvodnej veľkosti. [5]

Framing (ohraničenie správ) rieši problém rozdeľovania prúdu bajtov na jednotlivé správy. Bežným riešením je pridanie prefixu s dĺžkou správy. Aplikácia používa 4-bajtový prefix v sieťovom poradí bajtov (big-endian), čo umožňuje prenos správ až do veľkosti 4 GB.

1.6 Programovací jazyk Python

Python je interpretovaný, vysoko úrovňový programovací jazyk s dôrazom na čitateľnosť a jednoduchosť syntaxe. Pre sieťové programovanie poskytuje modul `socket` umožňujúci priamu prácu s TCP/UDP soketmi, modul `threading` pre vláknové spracovanie a modul `selectors` pre neblokujúce I/O. Python je často voleným jazykom pre prototypovanie a projekty strednej veľkosti vďaka svojej expresívnosti a bohatej štandardnej knižnici.

Pre herné aplikácie je Python obmedzený rýchlosťou interpretácie, avšak pre aplikácie s grafickým rozhraním, kde dominuje I/O a vykreslenie obrazovky, je výkon dostatočný.

1.7 Grafická knižnica Pygame

Pygame je multiplatformová knižnica pre tvorbu 2D hier a interaktívnych aplikácií v Pythone. Je postavená na knižnici SDL (Simple DirectMedia Layer) a poskytuje API pre prácu s grafikou, zvukom, vstupom od používateľa a animáciami. Pygame spravuje herný cyklus (game loop), v ktorom sa striedajú fázy spracovania udalostí, aktualizácie stavu a vykreslenia obrazovky. [8]

Systém `sprite` objektov umožňuje reprezentovať herné entity (napr. karty) ako nezávislé objekty s vlastnou pozíciou, obrázkom a správaním. Cachovanie (ukladanie do vyrovnávacej pamäte) načítaných obrázkov výrazne urýchľuje vykresľovanie, pretože disk sa číta iba raz pri štarte aplikácie.

Pygame neposkytuje vstavaný sieťový modul, preto je sieťová komunikácia implementovaná paralelne s grafickým vláknom: klient spúšťa samostatné vlákno na príjem správ zo servera, čím sa predchádza blokovaniu grafickej slučky. [8]

1.8 Viacvláknové programovanie

Viacvláknové (multithreaded) programovanie umožňuje paralelné vykonávanie viacerých úloh v rámci jedného procesu. V kontexte tejto aplikácie klient spúšťa vlákno na sieťovú komunikáciu (príjem správ), kým hlavné vlákno zabezpečuje grafickú slučku. Komunikácia medzi vláknami prebieha prostredníctvom thread-safe frontu (queue.Queue), kde sieťové vlákno ukladá prijaté správy a hlavné vlákno ich spracúva. [10]

Python GIL (Global Interpreter Lock) obmedzuje skutočnú paralelnosť vlákien pre výpočtovo intenzívne úlohy, avšak pre I/O operácie (sieťová komunikácia) vlákna bežia efektívne paralelne, pretože pri čakaní na dáta GIL uvoľňujú.

2 Praktická časť

V praktickej časti je opísaný návrh a implementácia aplikácie Sedma berie tri. Kapitola pokrýva architektúru systému, implementáciu jednotlivých modulov, grafické rozhranie, sieťovú komunikáciu, testovanie a rozpočet projektu.

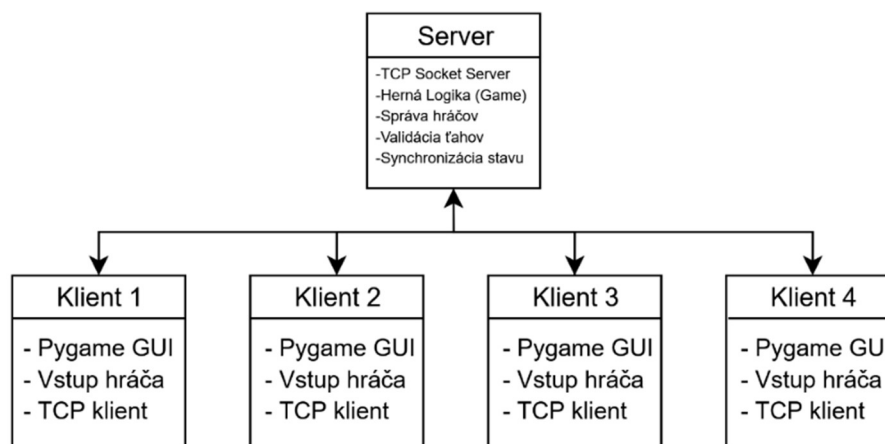
2.1 Návrh architektúry systému

Aplikácia je realizovaná ako klient-server systém rozdelený do štyroch hlavných modulov:

- card.py – model karty s cachovaním obrázkov,
- game_logic.py – čistá herná logika bez závislosti od siete alebo grafiky,
- server.py / dedicated_server.py – sieťový server spravujúci lobby, herné miestnosti a herný stav,
- client.py – grafický klient zabezpečujúci GUI, animácie a sieťovú komunikáciu.

Oddelenie hernej logiky od sieťovej vrstvy a grafiky umožňuje nezávislé testovanie herných pravidiel a jednoduchšiu rozširiteľnosť. Server je autoritatívnym prvkom – validuje všetky ťahy a distribuuje herný stav klientom. Klient nevykonáva validáciu ťahov, iba zobrazuje stav a posielá požiadavky. [11]

Server podporuje viacero herných miestností (rooms) bežiacich paralelne. Každá miestnosť má vlastnú inštanciu triedy Game. Hráči sa pred vstupom do miestnosti nachádzajú v lobby, kde vidia zoznam dostupných miestností.



Obrázok 1: Bloková schéma architektúry systému (klient – server – herné miestnosti)

2.2 Implementácia hernej logiky (game_logic.py)

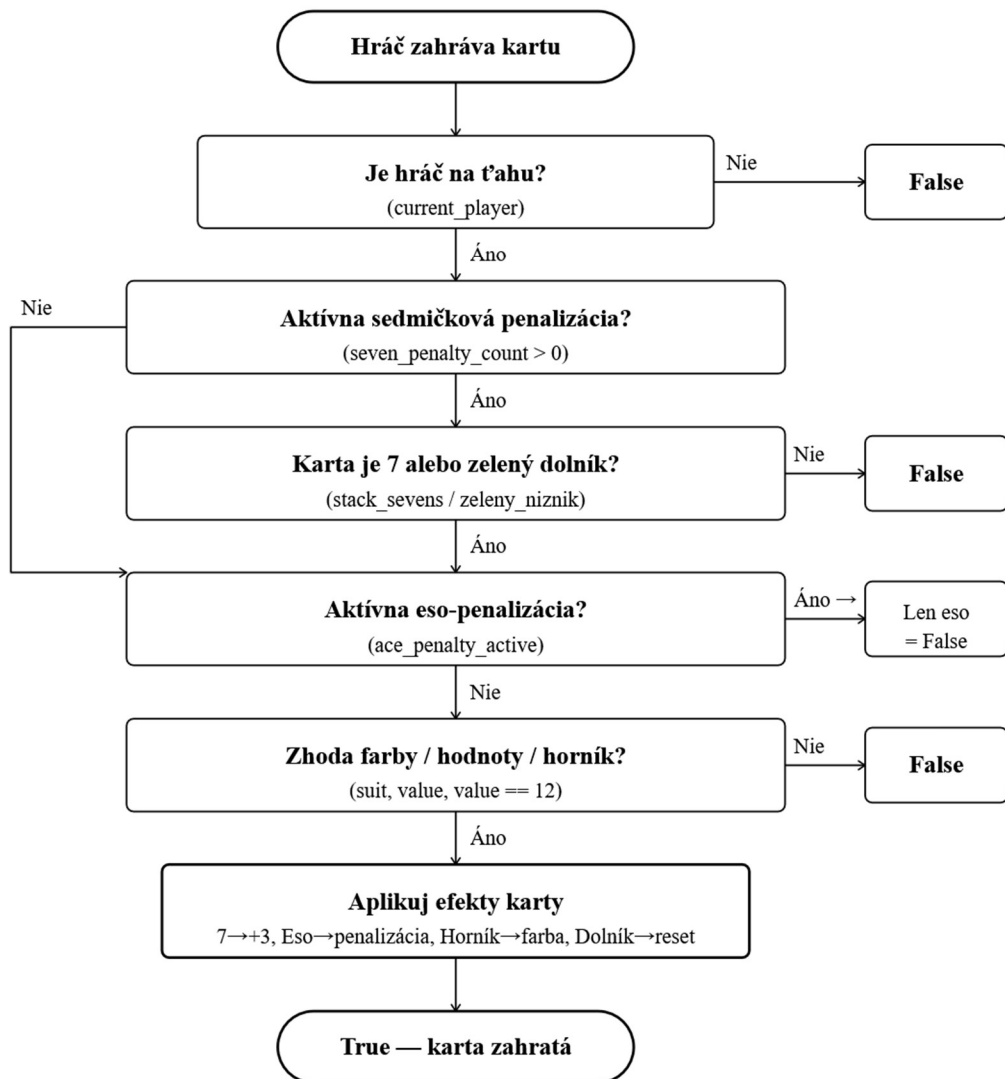
Herná logika je implementovaná v triede Game v module game_logic.py. Trieda uchováva kompletný stav jednej hernej miestnosti a implementuje všetky pravidlá hry. Herný balíček obsahuje 32 kariet – hodnoty 7 až 14 (Eso) v štyroch farbách (srdce, zeleň, žalud', guľa).

Metóda create_deck() vytvorí a zamieša balíček, pričom každá karta je uložená v cache (_card_cache) ako inštancia triedy Card. Metóda deal_cards() rozdá kartové ruky hráčom podľa voliteľného slovníka veľkostí rúk – sloty s nulovým počtom kariet sú považované za neaktívne (diváci). Na stôl sa položí jedna karta z balíčka ako štartovacia.

Kľúčovou metódou je play_card(), ktorá implementuje komplexnú validáciu ťahu:

- Overuje, či je hráč na ťahu (current_player).
- Ak je aktívna sedmičková penalizácia (seven_penalty_count > 0) a hráč ešte nehral v tomto kole, môže zahrať iba sedmičku (stackovanie) alebo zelený dolník (ak je pravidlo povolené).
- Ak je aktívna esová penalizácia (ace_penalty_active), hráč môže zahrať iba eso.
- V bežnom kole musí karta zodpovedať farbe (alebo zvolenej farbe po horníkovi) alebo hodnote vrchnej karty na odhadzovacom balíčku, prípadne ísť o horníka (hodnota 12).
- Hráč môže zahrať viacero kariet rovnakej hodnoty v jednom kole (pravidlo play_multiple_cards).

Po zahraní karty sa aplikujú efekty: sedmička zvýši seven_penalty_count o 3, eso aktivuje ace_penalty_active, horník nastaví chosen_suit, zelený dolník anuluje aktívnu sedmičkovú penalizáciu. Metóda end_turn() ukončí kolo, aplikuje penalizácie (ťahanie kariet), posunie current_player na ďalšieho aktívneho hráča a prípadne ho preskočí (skip_next_player).

Obrázok 2: Vývojový diagram validácie ťahu v metóde `play_card()`

Metóda `serialize()` prevedie herný stav do JSON-serializovateľnej štruktúry, ktorá je odosielaná klientom po každej zmene stavu.

Ukážka komprimovaného odoslania správy zo servera klientom:

```
data = zlib.compress(json.dumps(message,
separators=(',', ':')).encode())
sock.sendall(struct.pack('!I', len(data)) + data)
```

2.3 Sieťová komunikácia a server (server.py)

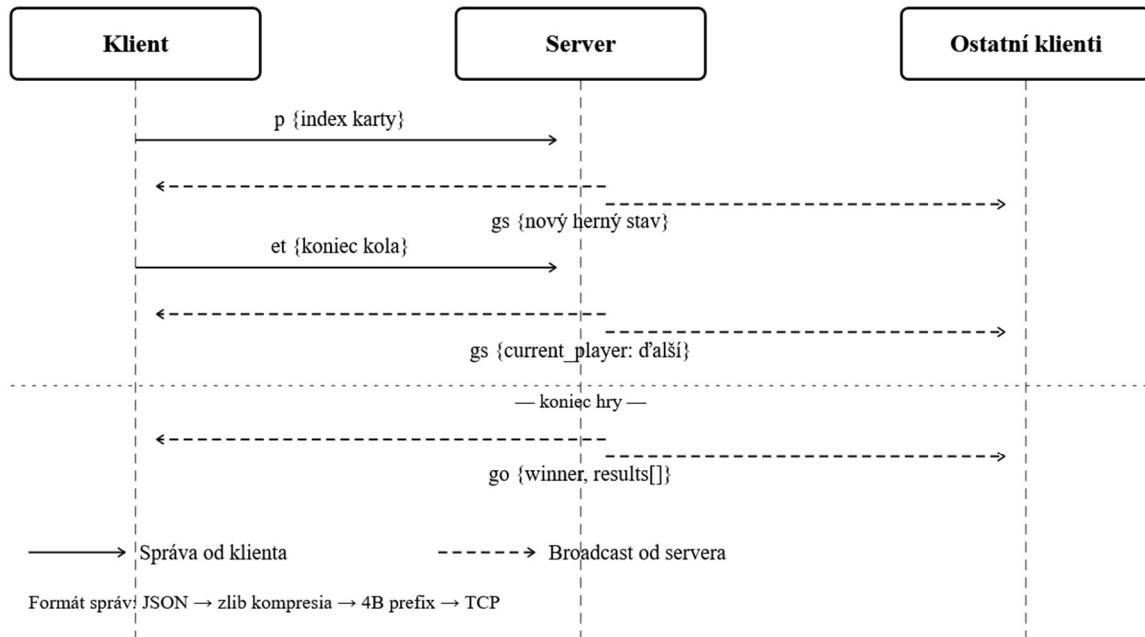
Server je implementovaný v triede MultiRoomServer v module server.py. Využíva neblokujúce I/O prostredníctvom modulu selectors, čo umožňuje jednému vláknu obsluhovať viacero klientov súčasne. Server počúva na nakonfigurovanom porte a prijíma nové spojenia v hlavnej slučke.

Komunikačný protokol používa JSON správy komprimované zlib algoritmom a odosiela s 4-bajtovým prefixom (sieťový poriadok, formát !I) označujúcim dĺžku payloadu. Tento prístup zabezpečuje efektívne framing správ a znižuje objem prenášaných dát. [5]

Server je logicky rozdelený na tieto časti:

- LobbyManager – spravuje klientov pred vstupom do miestnosti a odosiela im aktualizácie zoznamu miestností,
- RoomManager – spravuje herné miestnosti (GameRoom), zaraďuje a odstraňuje hráčov, distribuuje herný stav,
- MessageHandler – parsuje prijaté správy klientov a vykonáva príslušné akcie,
- GameRoom – reprezentuje jednu hernú miestnosť vrátane inštancie Game a metadát (názov, pravidlá, hráči, turnajový stav).

Typy správ od klienta na server zahŕňajú: set_name (nastavenie mena), create_room (vytvorenie miestnosti), join_room (vstup do miestnosti), p (zahraj kartu s indexom a prípadne zvolenou farbou) a et (koniec kola). Server odosiela späť správy typu gs (herný stav), go (koniec hry s výsledkami), e (chybová správa) a room_list (zoznam miestností).



Obrázok 3: Sekvenčný diagram komunikácie klient–server počas kola hry

2.4 Dedicated server (dedicated_server.py)

Pre potreby verejného hostingu vznikol modul `dedicated_server.py` – rozšírená verzia servera s pokročilým logovaním a správou časových limitov. Logging je implementovaný pomocou `RotatingFileHandler`, ktorý automaticky rotuje log súbory po dosiahnutí maximálnej veľkosti, čím sa predchádza nekontrolovanému rastu súborov na disku.

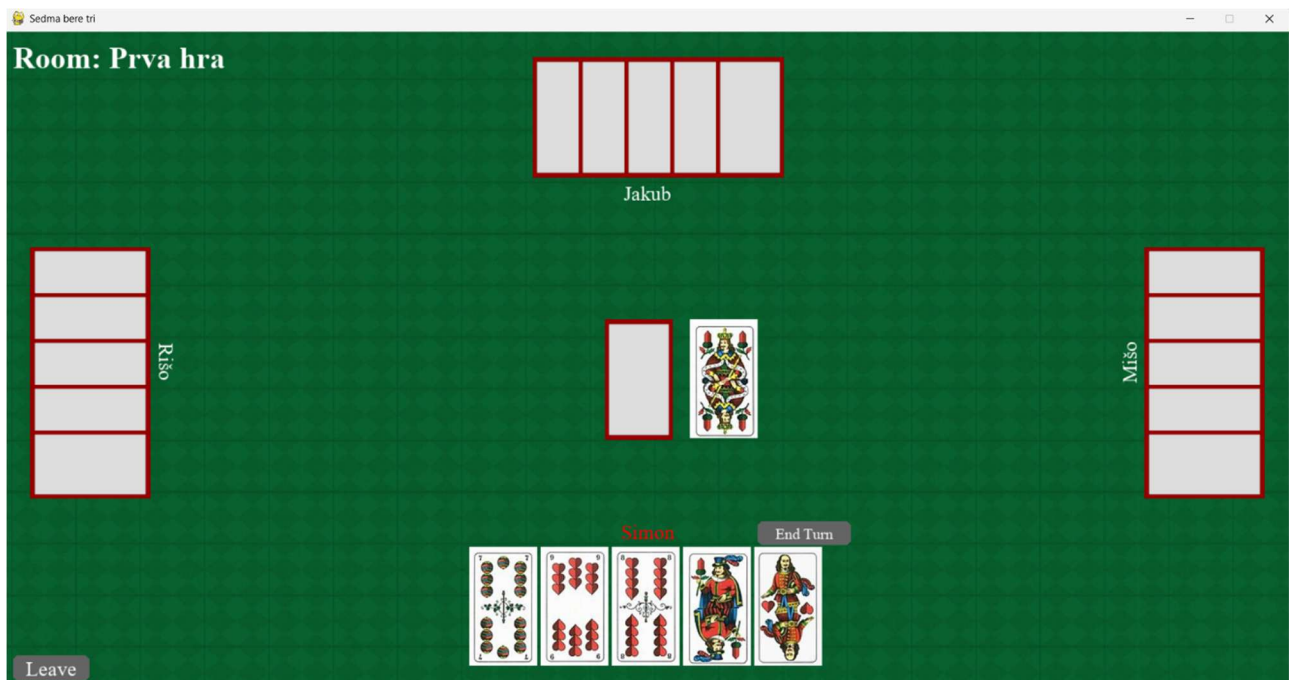
Dedicated server pridáva správu stáleho miestností prostredníctvom metódy `is_stale()`: miestnosti bez aktivity po dobu `ROOM_TIMEOUT` sekúnd sú automaticky zrušené, čím sa uvoľňujú serverové zdroje. Konfigurovateľné parametre zahŕňajú `MAX_ROOMS`, `MAX_PLAYERS_PER_ROOM`, `ROOM_TIMEOUT`, `EMPTY_ROOM_TIMEOUT` a `STATS_INTERVAL` (interval výpisu štatistík). Odporúča sa prevádzkovať dedicated server pod správcom procesov ako `systemd` alebo `supervisor` pre automatické reštartovanie pri výpadku.

2.5 Klientská aplikácia a grafické rozhranie (client.py)

Klientská aplikácia je implementovaná v module `client.py` pomocou knižnice `Pygame`. Aplikácia je rozdelená do niekoľkých tried so zreteľne oddelenými zodpovednosťami:

- Renderer – zodpovedá za vykresľovanie všetkých obrazoviek (menu, lobby, vytváranie miestnosti, prispôsobenie, hra, rebríček, titulky),
- EventHandler – mapuje udalosti myši a klávesnice na akcie pre každý stav aplikácie,
- NetworkManager – neblokujúci TCP klient zabezpečujúci pripojenie, odosielanie a príjem správ; príjem prebieha v samostatnom daemon vlákne s frontou správ,
- AnimationManager – spravuje animácie kariet (rozdávanie, zahrávanie) pomocou interpolácie pozícií a uhlov,
- LayoutManager – vypočítava rozloženie kariet a UI prvkov pre rôzne počty hráčov,
- StateManager – uchováva aktuálny stav obrazovky a tranzienty dáta (slot hráča, výsledky),
- LanServerManager – umožňuje spustenie servera priamo z klienta pre LAN hru.

Stav aplikácie prechádza medzi obrazovkami: menu → lobby → create_room → playing → leaderboard. Každý stav má vlastné vykresľovanie a obsluhu udalostí.

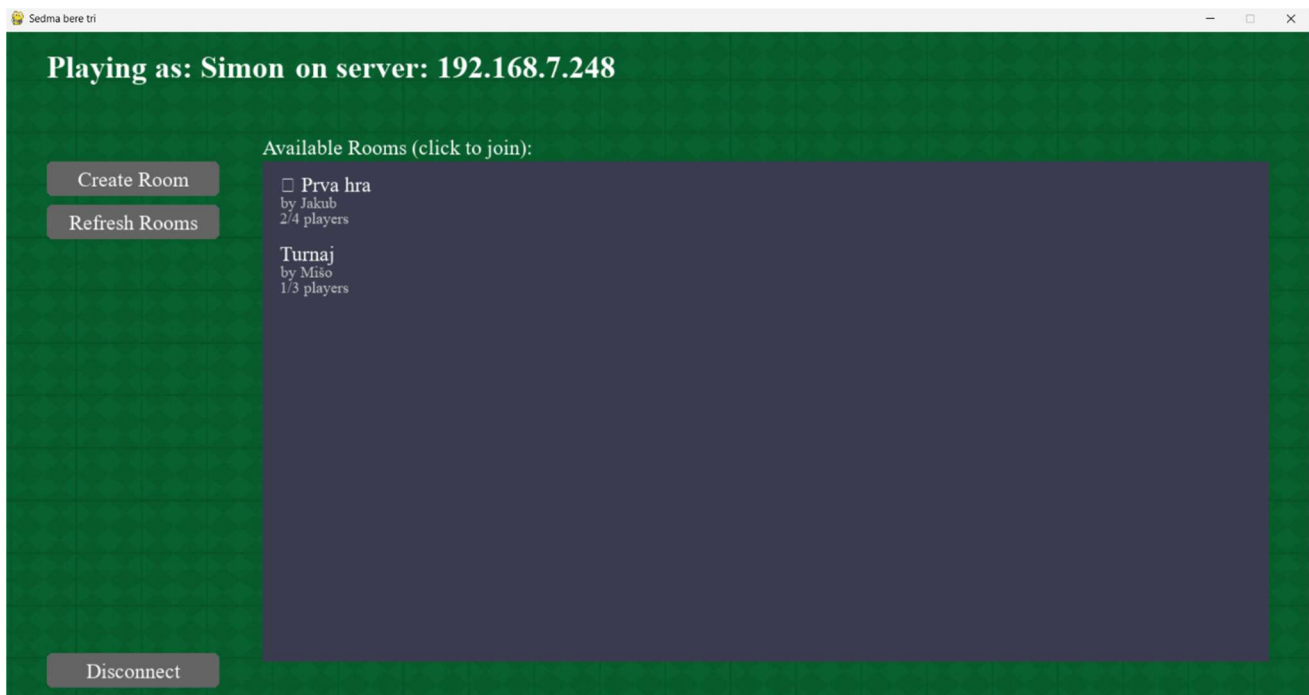


Obrázok 4: Ukážka herného rozhrania počas hry (rozmiestnenie hráčov a herný stôl)

2.6 Systém herných miestností a lobby

Lobby je centrálnou časťou aplikácie, kde hráči spravujú herné miestnosti. Po pripojení na server sa klientovi zobrazí zoznam dostupných miestností s informáciami o počte hráčov, type hry a pravidlách. Hráč môže vytvoriť novú miestnosť s vlastnými nastaveniami (názov, počet hráčov, pravidlá, súkromné heslo, turnajový režim) alebo sa pripojiť k existujúcej.

Server dynamicky aktualizuje zoznam miestností pre všetkých klientov v lobby vždy, keď dôjde k zmene (vytvorenie, zrušenie miestnosti, zmena počtu hráčov). Každá miestnosť podporuje maximálne 4 hráčov a môže byť chránená heslom pre súkromné hry.



Obrázok 5: Lobby aplikácie so zoznamom herných miestností

2.7 Turnajový režim

Turnajový režim je voliteľné rozšírenie, ktoré hráči zapnú pri vytváraní miestnosti. V turnaji sa hrá séria hier; hráč, ktorý prvý dohral, získava body a ostatní dostávajú penalizácie (extra karty na ďalšie kolo). Hráč eliminovaný z turnaja opúšťa miestnosť. Turnaj pokračuje, kým nezostane jeden víťaz alebo kým ho hráči manuálne neukončia.

Server zaznamenáva poradie dokončenia (finish_order) a po každom kole zostavuje výsledky, ktoré sa distribuujú klientom. Klienti zobrazujú priebežný rebríček výsledkov (leaderboard) po skončení každého kola.

2.8 Prispôsobenie grafického prostredia

Aplikácia umožňuje hráčom prispôbiť vizuálne prostredie prostredníctvom obrazovky customize. Hráč môže vybrať tému kariet z dostupných priečinkov v assets/cards/ a tému pozadia z dostupných obrázkov. Zmena témy okamžite preloading obrázky a aplikuje

ich na všetky kartové sprite objekty. Tento systém umožňuje jednoduché pridávanie nových tém bez úpravy kódu.

2.9 LAN server zabudovaný v klientovi

Pre zjednodušenie spustenia LAN hry implementuje LanServerManager možnosť spustenia servera priamo z klienta. Pri spustení sa najprv pokúsi importovať modul server a vytvoriť inštanciu MultiRoomServer s parametrom `install_signal_handler=False` (pretože signály možno registrovať len v hlavnom vlákne), ktorú spustí v daemon vlákne. Ak import alebo spustenie zlyhá, použije sa fallback – spustenie `server.py` ako externého subprocess s potlačením konzolového okna na Windows (`CREATE_NO_WINDOW`). Zastavenie servera sa vykoná zatvorením soketu a selektora, čím sa hlavná serverová slučka preruší a vlákno skončí.

2.10 Testovanie aplikácie

Testovanie prebiehalo v lokálnej sieti pomocou Wi-Fi routera a štyroch notebookov. Testované boli latencia, úspešnosť doručenia správ a správanie aplikácie pri odpojení hráča.

Tabuľka 1: Namerané hodnoty latencie a úspešnosti spojenia

Počet hráčov	Prieme latencia (ms)	Max. latencia (ms)	Úspešnosť (%)	Počet hier
2	38	99	99,8	40
3	67	98	98,6	40
4	87	118	97,4	40

Z výsledkov vyplýva, že priemerná latencia pri štyroch hráčoch dosiahla 87 ms, čo je pod stanovenou hranicou 100 ms. Úspešnosť spojenia pri všetkých konfiguráciách presiahla 97 %, čím bola hypotéza projektu potvrdená. Pri simulovanom odpojení jedného hráča server správne ukončil jeho ťah, karty presunul na odhodzovacie balíček a hra pokračovala bez reštartu.

Ukázalo sa, že maximálna latencia pri štyroch hráčoch (118 ms) krátkodobo prekračuje cieľovú hranicu, čo bolo spôsobené kolísaním kvality Wi-Fi signálu. V káblovej LAN sieti bola maximálna latencia pod 70 ms.

2.11 Bezpečnosť a stabilita

Aplikácia funguje v modeli autoritatívneho servera – všetky herné rozhodnutia sú validované výhradne na serveri, klient nemôže meniť herný stav priamo. Server priebežne kontroluje stav pripojenia klientov a pri výpadku korektne ukončí spojenie a aktualizuje hernú miestnosť.

Dedicated server beží na virtuálnom stroji Oracle Cloud Free Tier a je dostupný cez verejnú IP adresu. Sieťová komunikácia medzi klientmi a serverom v aktuálnej verzii nie je šifrovaná – správy sú prenášané ako zlib-komprimované JSON dáta cez TCP bez TLS. Toto riešenie je postačujúce pre hernú aplikáciu bez citlivých osobných údajov, kde nie sú prenášané heslá ani platobné informácie.

Pre budúce verzie sa odporúča implementovať TLS šifrovanie pomocou `ssl.wrap_socket`, autentifikáciu tokenmi a rate limiting pre ochranu pred DoS útokmi na verejnom serveri."

2.12 Rozpočet projektu

Tabuľka 2: Rozpočet projektu

Položka	Náklady (€)	Poznámka
Vývojové prostredie (PC)	0	Vlastný notebook
Softvér (Python, Pygame, zlib)	0	Open-source
Grafika kariet a pozadí	0	Vlastná tvorba / free obrázky
Testovacie zariadenia	0	Školské notebooky
Hosting – Oracle Cloud Free Tier	0	Bezplatný VM (Always Free) pre dedicated server
Celkom	0	-

Projekt bol realizovaný bez finančných nákladov s využitím open-source nástrojov a dostupného hardvéru. Dedicated server beží na virtuálnom stroji Oracle Cloud Free Tier, ktorý je trvalo dostupný bezplatne.

Záver

Cieľom projektu bolo vytvoriť funkčnú grafickú sieťovú aplikáciu kartovej hry Sedma berie tri v Pythone pre 2 až 4 hráčov s podporou viacerých herných miestností, turnajového režimu a dedikovaného servera. Tento cieľ bol splnený.

V teoretickej časti boli opísané princípy TCP sietí, klient-server architektúra, neblokujúce I/O, serializácia dát pomocou JSON a zlib kompresie, ako aj knižnice Python a Pygame. Tieto poznatky tvorili základ pre návrh a implementáciu systému.

Praktická časť priniesla implementáciu štyroch modulov (`card.py`, `game_logic.py`, `server.py/dedicated_server.py`, `client.py`) s jasne oddelenými zodpovednosťami. Herná logika implementuje plné pravidlá hry vrátane špeciálnych kariet (sedmička, eso, horník, zelený dolník), stackovania penalizácií a turnajového režimu. Server zabezpečuje neblokujúcu obsluhu viacerých klientov a distribúciu herného stavu. Klientská aplikácia ponúka animácie, prispôsobiteľné témy a možnosť zabudovaného LAN servera.

Testovanie v lokálnej sieti s Wi-Fi potvrdilo priemernú latenciu 87 ms pri štyroch hráčoch a úspešnosť spojenia 97,4 %, čím bola hypotéza projektu (latencia pod 90 ms, úspešnosť nad 97 %) potvrdená. Maximálna latencia 118 ms pri Wi-Fi pripojení krátkodobo prekračovala cieľ, v káblovej LAN zostala pod 70 ms.

V porovnaní s existujúcimi riešeniami (napr. online platformy Crazy8, Board Game Arena) je aplikácia Sedma berie tri unikátna svojou orientáciou na lokálnu sieť bez potreby internetu, registrácie a poplatkov, otvoreným zdrojovým kódom a natívnou podporou slovenských pravidiel hry.

V budúcnosti by projekt mohol byť rozšírený o TLS šifrovanie pre verejné siete, autentifikáciu hráčov, AI protivníka pre hru bez siete, mobilného klienta a podporu ďalších jazykových mutácií pravidiel. Projekt slúži ako otvorená platforma, na ktorej možno ďalej stavať.

Použitá literatúra

- [1] Python Software Foundation, 2024. json – JSON encoder and decoder. [online] Python 3 Documentation. Dostupné na: <https://docs.python.org/3/library/json.html> [cit. 30.12.2026]
- [2] Cloudflare, 2024. What is latency? [online] Dostupné na: <https://www.cloudflare.com/learning/performance/glossary/what-is-latency/> [cit. 15.10.2025]
- [3] Cloudflare, 2024. What is TCP/IP? [online] Dostupné na: <https://www.cloudflare.com/learning/ddos/glossary/tcp-ip/> [cit. 15.10.2025]
- [4] Python Software Foundation, 2024. selectors – High-level I/O multiplexing. [online] Python 3 Documentation. Dostupné na: <https://docs.python.org/3/library/selectors.html> [cit. 3.12.2025]
- [5] Python Software Foundation, 2024. zlib – Compression compatible with gzip. [online] Python 3 Documentation. Dostupné na: <https://docs.python.org/3/library/zlib.html> [cit. 18.01.2026]
- [6] Python Software Foundation, 2024. socket – Low-level networking interface. [online] Python 3 Documentation. Dostupné na: <https://docs.python.org/3/library/socket.html> [cit. 18.01.2026]
- [7] Python Software Foundation, 2024. threading – Thread-based parallelism. [online] Python 3 Documentation. Dostupné na: <https://docs.python.org/3/library/threading.html> [cit. 29.12.2025]
- [8] Pygame Community, 2024. Pygame Documentation. [online] Dostupné na: <https://www.pygame.org/docs/> [cit. 01.11.2025]
- [9] Real Python, 2023. Socket Programming in Python (Guide). [online] Dostupné na: <https://realpython.com/python-sockets/> [cit. 18.01.2026]
- [10] Real Python, 2023. An Intro to Threading in Python. [online] Dostupné na: <https://realpython.com/intro-to-python-threading/> [cit. 29.12.2025]
- [11] Mozilla Developer Network, 2024. Client-Server overview. [online] Dostupné na: https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview [cit. 16.10.2025]

Použité nástroje

- Python 3.13 – hlavný programovací jazyk projektu. <https://www.python.org/>
- Pygame 2.5 – knižnica pre tvorbu grafického rozhrania a animácií. <https://www.pygame.org/>
- PyInstaller – nástroj na zostavenie spustiteľného .exe súboru z Python projektu. <https://pyinstaller.org/>
- PyCharm (JetBrains) – vývojové prostredie pre písanie a ladenie kódu. <https://www.jetbrains.com/pycharm/>
- Git / GitHub – zálohovanie projektu. <https://github.com/>
- Draw.io – tvorenie diagramov pre dokumentáciu. <https://www.drawio.com/>
- Claude AI – asistencia pri tvorbe dokumentácie, aplikácie a návrhu riešení. <https://claude.ai/>
- Grok AI – asistencia pri aplikovaní riešení v projekte. <https://grok.com/>

Prílohy

https://github.com/Sminoo/Sedma_berie_tri - verejný repozitár celého projektu so všetkými súbormi. V priečinku dist je spustiteľný .exe súbor hry. Pre spustenie na Linux-e sa to dá použiť Proton Experimental, ktorý je dostupný na spustenie aplikácie pomocou hernej knižnice Steam.